SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct
I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: _____

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud.
   Discuss how cloud storage solutions ensure durability and data redundancy.
   How can encryption and access control protect sensitive backup data?
   Explain the cost advantages compared to traditional storage systems.
   Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
   Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

Question 1: Auto-Scaling and Load Balancing for Seasonal Traffic

Question: ***A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.***

1.1 Understanding the Problem

Seasonal sales such as Diwali, Black Friday, Amazon Prime Day or the Flipkart Big Billion Days create traffic that is very different from normal days. Traffic can grow ten to fifty times within minutes of a flash-sale announcement and then fall sharply a few hours later. These spikes are unpredictable in their exact timing and size, because they depend on marketing campaigns, social media buzz and competitor activity.

In a traditional data centre the company has only two options. It can buy enough servers for the peak, which leaves most of the hardware idle for eleven months of the year, or it can buy for average load and accept slow pages and crashes during the sale. Both options are poor. The first wastes capital and the second damages revenue and brand reputation. Cloud computing removes this dilemma through elasticity, which means resources can be added or removed automatically as demand changes, and the customer pays only for what is used.

Two cloud services work together to deliver this elasticity. Auto-scaling adjusts the number of compute resources, and load balancing distributes incoming requests across those resources. Neither one is sufficient alone. Auto-scaling without a load balancer would create new servers that receive no traffic, and a load balancer without auto-scaling would spread traffic evenly across a fixed number of servers that are all overloaded.

1.2 Cloud Auto-Scaling

Auto-scaling is a cloud feature that automatically increases or decreases the number of virtual machines, containers or function instances in a group based on rules defined by the administrator. It is offered as Amazon EC2 Auto Scaling, Azure Virtual Machine Scale Sets and Google Compute Engine Managed Instance Groups.

Types of scaling

- **Horizontal scaling (scale out and scale in): adding or removing instances. This is the preferred method for web applications because there is no practical upper limit and a failed instance does not stop the service.**
- **Vertical scaling (scale up and scale down): moving to a larger or smaller instance type. It usually needs a restart and is limited by the largest machine available, so it suits databases more than web servers.**

Scaling policies

Policy type	How it works	E-commerce example
Target tracking	Keeps a metric near a chosen value, for example 55% average CPU	Add servers automatically when CPU on the product-page fleet rises above 55%
Step scaling	Adds a different number of instances depending on how far the alarm is breached	Add 4 instances at 70% CPU, add 12 instances at 90% CPU
Policy type	How it works	E-commerce example

Scheduled scaling	**Changes capacity at a fixed date and time**	**Raise the minimum from 4 to 30 instances at 11:45 PM before a midnight sale**
Predictive scaling	**Uses machine learning on historical load to forecast demand**	**Learns last year's festive pattern and pre-warms capacity**

Main components

- **Launch template or machine image: a pre-configured definition of the instance (operating system, application code, security groups) so every new server is identical.**
- **Auto-scaling group: the logical group with a minimum, desired and maximum size, spread across several Availability Zones.**
- **Scaling policies and alarms: the rules that decide when to scale.**
- **Cooldown and warm-up periods: short waiting times that stop the system from adding and removing servers too rapidly (called thrashing).**

1.3 Cloud Load Balancing

A load balancer is a managed service that sits in front of the servers and acts as a single entry point for all user requests. It accepts each request and forwards it to a healthy back-end instance. Examples are AWS Elastic Load Balancing, Azure Load Balancer and Application Gateway, and Google Cloud Load Balancing.

- **Layer 4 (network) load balancing routes traffic using IP address and port. It is very fast and suited for high-volume TCP and UDP traffic.**
- **Layer 7 (application) load balancing understands HTTP and can route by URL path or header. For example, requests to /checkout can go to one server group and /search to another.**
- **Health checks are small periodic requests sent to every instance. A server that fails the check is removed from rotation automatically, so users never reach a broken server.**
- **Session persistence and stateless design: sticky sessions can bind a user to one server, but a better design stores the shopping cart in a shared cache such as Redis so any server can serve any user.**
- **Multi-zone distribution: the balancer spreads traffic across several Availability Zones, so the loss of one data centre does not stop the shop.**

Algorithm	Behaviour	Best used when
Round robin	**Sends requests to servers in turn**	**Servers are identical and requests are similar**
Least connections	**Sends the request to the server with the fewest active connections**	**Requests have very different durations, such as checkout versus browsing**
Weighted	**Gives more traffic to stronger servers**	**The fleet mixes large and small instance types**
IP hash	**Same client always reaches the same server**	**Session affinity is required**

1.4 How the Two Services Work Together

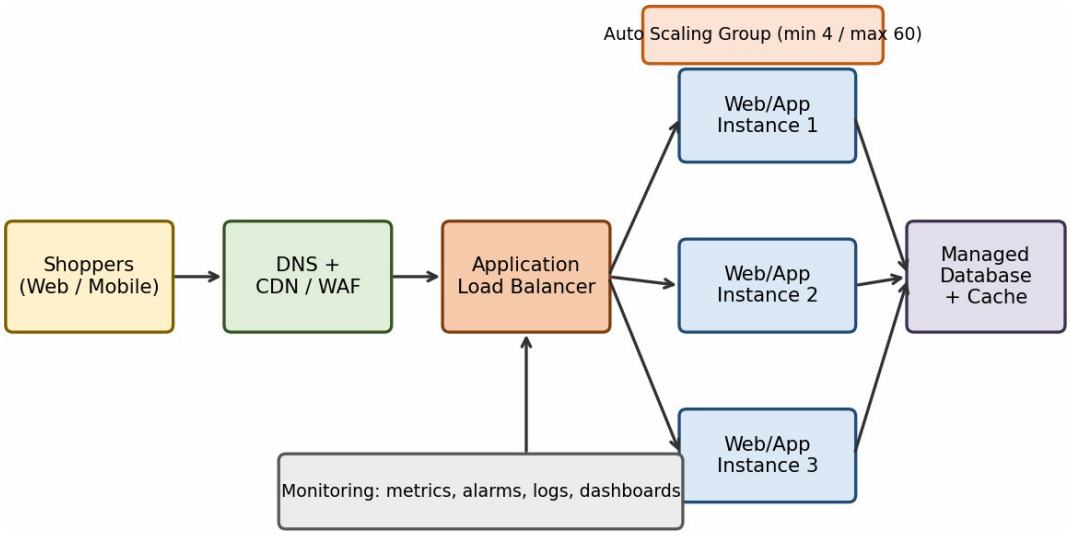


Figure 1: Auto-scaling with load balancing for an e-commerce platform

The flow in a typical sale event is as follows. A shopper's request reaches the DNS service and passes through the CDN and web application firewall. The load balancer then forwards it to one of the healthy instances in the auto-scaling group. As more shoppers arrive, average CPU or request count rises. A monitoring alarm fires, the auto-scaling group launches new instances from the template, the load balancer detects them through its health checks and begins sending them traffic. When the sale ends and the metric falls, surplus instances are drained and terminated, so the company stops paying for them.

1.5 Reducing Downtime and Improving User Experience

Benefit	How it is achieved
No single point of failure	**Many instances across several Availability Zones sit behind the balancer. If one instance or even one zone fails, the others continue.**
Self-healing	**Auto-scaling replaces unhealthy instances automatically to maintain the desired capacity.**
Consistent response time	**Capacity grows before queues build up, so pages load in about the same time at 2 PM on a normal day and 12:05 AM in a sale.**
Zero-downtime deployments	**Rolling or blue-green updates let new versions be introduced gradually while the balancer drains old servers.**
Protection from overload	**Connection draining, rate limiting and health checks prevent a slow server from receiving more work.**
Cost efficiency	**The company pays for sale capacity only during the sale hours, which protects margins.**

From the shopper's side the result is simple: search results appear quickly, the cart does not lose items, payment pages do not time out and the site remains available. Studies on e-commerce behaviour regularly show that each additional second of page delay increases cart abandonment, so stable performance during peak events directly protects revenue.

1.6 Role of Monitoring Tools

Auto-scaling is only as good as the data that drives it. Monitoring tools collect, analyse and act on that data. Amazon CloudWatch, Azure Monitor, Google Cloud Operations, Prometheus with Grafana, Datadog and New Relic are commonly used for this purpose.

- **Metrics collection: CPU utilisation, memory, request count per target, response latency, error rate (HTTP 5xx) and queue depth are gathered every few seconds to a minute.**
- **Alarms and triggers: when a metric crosses a threshold, the alarm triggers the scaling policy or notifies the on-call engineer.**
- **Dashboards: real-time views let the operations team watch orders per minute, latency and healthy host count during the sale.**
- **Log analysis and tracing: centralised logs and distributed tracing show which microservice is slow, for example the payment gateway or inventory service.**
- **Synthetic monitoring: scripted tests that simulate a user adding an item and paying detect failures before real customers do.**
- **Capacity planning and cost reports: historical data help the team set better minimum capacity and budget for the next sale.**

1.7 Real-World E-Commerce Scenario

Consider an online fashion retailer, which we call ShopKart, running a 24-hour Diwali sale that starts at midnight. On a normal day the platform needs 6 web servers. The team configures an auto-scaling group with a minimum of 6 and a maximum of 80 instances, a target-tracking policy at 55% CPU and an Application Load Balancer across three Availability Zones. A scheduled action raises the minimum to 30 at 11:45 PM so the first wave does not hit cold servers.

Time	Event	System response
11:45 PM	Scheduled scaling increases the minimum to 30	30 instances boot and pass health checks
12:00 AM	Sale starts, requests grow 25 times in ten minutes	Target tracking adds 25 more instances, total 55
12:20 AM	One instance in zone B crashes	Health check removes it, auto-scaling launches a replacement, users see no error
2:00 AM	Traffic stabilises	Fleet holds at about 55 instances
10:00 AM	Second flash offer announced	Alarm triggers step scaling to 70 instances
12:30 AM next day	Sale ends	Scale-in gradually returns to 6 instances, costs fall

In this scenario the retailer serves the sale without a crash, pays for the extra servers only for roughly a day and keeps customers happy. Real companies such as Netflix, Flipkart and Amazon use the same principles at much larger scale.

1.8 Challenges and Best Practices

- **Design the application to be stateless, storing sessions in a shared cache and files in object storage.**
- **Use pre-baked machine images or containers to reduce start-up time, and keep a warm pool of ready instances for sudden spikes.**

- **Scale the database tier with read replicas, caching and managed services, because the database is often the real bottleneck.**
- **Set maximum limits and budget alerts so a bot attack or a faulty policy does not create unlimited cost.**
- **Run load tests before the season to verify that the scaling rules respond quickly enough.**

1.9 Summary

Auto-scaling gives the platform the right amount of compute at every moment, and load balancing makes sure that compute is used evenly and safely. Monitoring ties the two together by providing the signals that trigger scaling and by giving the team visibility. Together they reduce downtime, deliver consistent response times and make seasonal sales both reliable and economical.

Question 2: Secure and Cost-Effective Cloud Backup Storage

Question: ***An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.***

2.1 Introduction

Backup is the last line of defence against hardware failure, human error, ransomware, software bugs and natural disasters. As organisations generate terabytes or even petabytes of data, maintaining tape libraries, disk arrays and off-site vaults becomes expensive and slow to manage. Cloud storage services such as Amazon S3 and S3 Glacier, Azure Blob Storage and Google Cloud Storage provide a scalable, managed alternative in which the provider takes responsibility for the physical infrastructure while the customer controls the data, the keys and the policies.

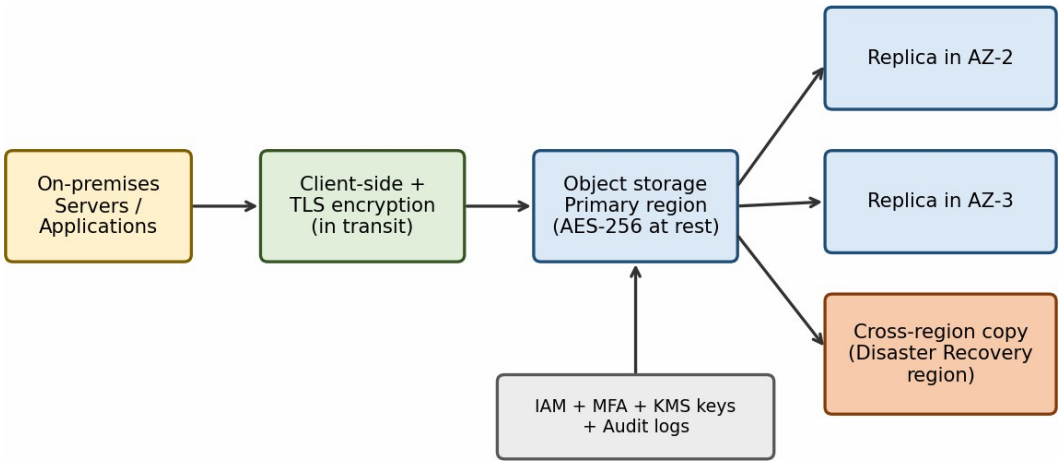


Figure 2: Secure, redundant cloud backup architecture

2.2 Durability and Data Redundancy

Durability is the probability that stored data will not be lost over time, whereas availability is the probability that data can be accessed at a given moment. Backups need very high durability above all. Leading providers advertise durability of 99.999999999% (eleven nines) for their standard object storage classes. This means that if a customer stored ten million objects, the expected loss would be about one object in ten thousand years.

This level of durability is achieved through several design techniques:

- **Replication across devices and facilities: every object is stored on multiple disks in at least three separate Availability Zones within a region. Each zone has independent power, cooling and networking.**
- **Erasure coding: data is split into fragments and extra parity fragments are calculated. The original object can be rebuilt even if several fragments are lost, using less space than keeping full copies.**
- **Checksums and integrity checking: the service continuously verifies stored objects using checksums and automatically repairs any corrupted copy from a good copy (this is called scrubbing or self-healing).**
- **Cross-region replication: objects can be copied automatically to a second geographic region to survive the loss of a whole region.**
- **Versioning and object lock: every change creates a new version, and write-once-read-many (WORM) locking makes backups immutable so that ransomware or an accidental delete cannot destroy them.**

Redundancy option	Protects against	Typical use
Locally redundant (LRS)	Disk or rack failure within one data centre	Low-value, easily recreated data
Zone redundant (ZRS / multi-AZ)	Failure of a complete data centre	Production backups
Geo redundant (GRS / cross-region)	Regional disaster such as flood or large outage	Critical backups and disaster recovery
Geo-zone redundant (GZRS)	Both of the above combined	Highest protection for regulated data

2.3 Storage Classes and Lifecycle Management

Backups are written often but read rarely, so cloud providers offer tiered storage classes. Recent backups sit in a hot or standard tier for quick restore, older backups move to cool or infrequent-access tiers and long-term archives move to archive tiers such as S3 Glacier Deep Archive or Azure Archive. Lifecycle rules perform these transitions automatically. For example: keep daily backups in the standard tier for 30 days, move them to infrequent access until 90 days, then archive them for seven years and finally delete them. This policy reduces cost while meeting retention requirements.

2.4 Protecting Backup Data with Encryption

Encryption converts readable data into ciphertext that is useless without the correct key. Protection is needed at three stages.

- **Encryption in transit: data travels over TLS 1.2 or higher, or through private connections such as AWS Direct Connect or Azure ExpressRoute, so that it cannot be read or altered on the network.**
- **Encryption at rest: objects are encrypted with AES-256 before they are written to disk. This can be done with provider-managed keys, customer-managed keys in a Key Management Service (KMS), or customer-supplied keys.**

- **Client-side encryption: the organisation encrypts data before upload, so the provider never sees plaintext. This gives the highest level of control and is common for very sensitive data.**
- **Key management: keys are stored in KMS or hardware security modules (HSM), rotated regularly and protected by separate permissions. Deleting a key makes the associated data permanently unreadable, which is useful for secure disposal (crypto-shredding).**

2.5 Protecting Backup Data with Access Control

Encryption protects the data itself and access control decides who can reach it. Cloud platforms apply the principle of least privilege, giving each user or application only the permissions needed for its task.

- **Identity and Access Management (IAM): roles and policies specify who can read, write or delete backup buckets. A backup application, for example, may be allowed to write but not delete.**
- **Multi-factor authentication (MFA): adds a second proof of identity. Many providers support MFA-delete, requiring MFA before a version or bucket can be removed.**
- **Role-based access control (RBAC) and separation of duties: backup administrators, security staff and auditors have different roles, so no single person can alter and hide data.**
- **Network controls: private endpoints, VPC or VNet restrictions and IP allow-lists prevent public exposure. Public access blocking should be enabled by default.**
- **Logging and auditing: services such as CloudTrail and Azure Activity Log record every access attempt, and alerts highlight unusual activity such as mass deletion.**
- **Immutable and air-gapped copies: a separate account with restricted credentials holds a locked copy that cannot be modified even by a compromised administrator.**

2.6 Cost Advantages over Traditional Storage

Traditional backup requires buying disk arrays or tape libraries, leasing off-site space, paying for power, cooling, administrators and media transport, and periodically replacing hardware. Capacity must be bought in advance, so utilisation is low. The cloud turns this into an operating expense that scales with use.

Cost factor	Traditional on-premises backup	Cloud backup storage
Upfront investment	High capital cost for arrays, tape drives and licences	Almost none; pay as you go
Capacity planning	Buy for the future, so spare capacity sits unused	Elastic; grows and shrinks with data
Off-site copy	Second data centre or courier service for tapes	Built-in multi-zone and cross-region replication
Maintenance and staff	Hardware refresh, patching, media handling	Managed by the provider
Power and cooling	Borne by the organisation	Included in the price
Archive cost	Tape handling is cheap per GB but slow to retrieve	Archive tiers cost a fraction of a cent per GB per month
Recovery speed	Hours or days to retrieve tapes	Minutes to hours depending on tier
Total cost of ownership	High and unpredictable	Lower and predictable; costs can be tagged and budgeted

The organisation should still plan for charges such as data retrieval from archive tiers, early-deletion fees and data egress. Using lifecycle rules, compression, deduplication and incremental backups keeps the bill under control.

2.7 Disaster Recovery Use Case

Consider a hospital group in Chennai that stores patient records, imaging files and billing databases. It uses nightly full backups and hourly incremental backups, encrypted with customer-managed keys and written to cloud storage in the Mumbai region. The bucket has versioning and object lock, and objects are replicated automatically to the Hyderabad region.

Step	What happens
Incident	A ransomware attack encrypts the primary file servers and the local backup appliance.
Detection	Monitoring alerts report mass file modification. The security team isolates the network.
Recovery decision	The team chooses the last clean restore point from before the attack. Immutable versions guarantee that the cloud copies are untouched.
Restoration	Virtual machines are launched in the cloud DR region from backup images, databases are restored from the latest clean snapshot and DNS is switched to the recovered environment.
Result	Critical systems return within a few hours (RTO) and data loss is limited to the last hour (RPO). Normal operation later fails back to the rebuilt on-premises systems.

Two metrics guide such plans. The Recovery Time Objective (RTO) is how quickly service must be restored, and the Recovery Point Objective (RPO) is how much data loss is acceptable. Cloud backup, snapshots and replication allow organisations to choose strategies from simple backup and restore (low cost, hours of recovery) to pilot light, warm standby and multi-site active-active (higher cost, near-zero downtime).

2.8 Best Practices

- **Follow the 3-2-1 rule: three copies of data, on two different media types, with one copy off-site or in another region.**
- **Test restores regularly, because an untested backup cannot be trusted.**
- **Enable versioning, object lock and MFA-delete for protection against ransomware and mistakes.**
- **Encrypt everything and manage keys separately from data.**
- **Define retention policies that meet legal requirements and avoid keeping data longer than necessary.**

2.9 Summary

Cloud storage gives backup data extreme durability through replication, erasure coding and continuous integrity checks. Encryption and strict access control keep that data confidential and tamper-proof. Compared with traditional systems, the cloud removes large upfront costs, provides built-in off-site protection and speeds up recovery, making it the preferred platform for modern backup and disaster recovery.

Question 3: Global Deployment with Content Delivery Networks

Question: ***A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.***

3.1 The Challenge of Global Delivery

When a mobile application is hosted in a single location, every user must send requests to that location. A user in Chennai connecting to a server in Virginia must cross undersea cables and many network hops. The time for a data packet to make a round trip is called latency, and it is limited by the speed of light in fibre, by the number of routers on the path and by congestion. Even at best the round trip between India and the US East coast takes around 200 to 250 milliseconds, which is enough to make videos buffer, web pages feel slow and games unplayable.

The app company therefore needs to bring content and compute physically close to users. This is the role of Content Delivery Networks and the global infrastructure of cloud providers.

3.2 What Is a CDN?

A CDN is a geographically distributed network of servers, called edge servers or Points of Presence (PoPs), that store copies of content close to end users. Popular CDNs include Amazon CloudFront, Azure Front Door and CDN, Google Cloud CDN, Cloudflare and Akamai. A user's request is routed to the nearest or best-performing edge location instead of travelling to the origin server.

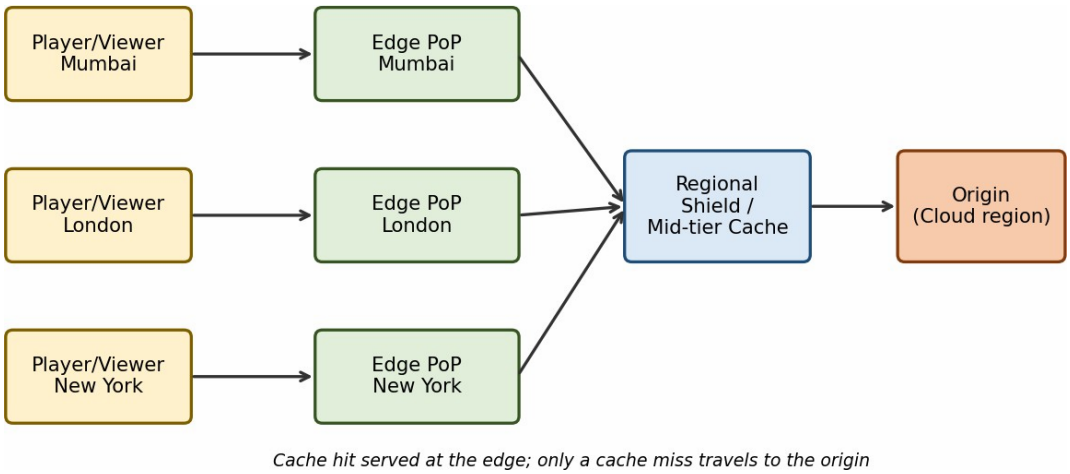


Figure 3: Global content delivery through CDN edge servers

How a CDN request works

- **The user opens the app, and the app requests a video segment, image or API response from a domain such as cdn.example.com.**
- **DNS or Anycast routing directs the request to the closest healthy edge location.**
- **If the edge has the object in cache (a cache hit), it returns it immediately, usually within a few milliseconds.**

- **If not (a cache miss), the edge fetches the object from a regional cache or the origin, stores a copy according to the cache-control headers and serves the user. Later requests from nearby users become cache hits.**
- **The edge keeps persistent, optimised connections to the origin, which makes even cache misses faster than a direct connection.**

3.3 How CDNs Improve Performance and Reduce Latency

Technique	Explanation	Effect
Geographic proximity	Content is served from a nearby PoP	Round-trip time falls from hundreds of milliseconds to tens
Caching	Static assets such as images, scripts, video segments and app updates are stored at the edge	Fewer requests to the origin, less server load
Connection optimisation	TLS termination at the edge, HTTP/2 and HTTP/3, TCP tuning and connection reuse	Faster handshakes and higher throughput
Compression and image optimisation	Gzip or Brotli and adaptive image formats	Smaller payloads on mobile networks
Load distribution	Traffic is spread across many edge servers	Handles viral spikes without origin overload
Dynamic acceleration	Optimised routing for non-cacheable API calls	Lower latency even for personalised content
DDoS and WAF protection	Attack traffic is absorbed at the edge	Origin stays available and secure

3.4 How Edge Servers Enhance User Experience across Regions

Edge servers are the visible face of the CDN. They provide benefits to users in every region, including those with weaker networks.

- **Faster loading: app screens, images and updates are downloaded from a nearby server, reducing wait times.**
- **Smooth video streaming: with adaptive bitrate streaming (HLS or MPEG-DASH), video is divided into short segments cached at the edge. The player selects the quality that fits the user's bandwidth, so there is less buffering.**
- **Consistency across regions: a user in Mumbai, London or São Paulo gets nearly the same experience because each is served locally.**
- **Reliability: if one edge location fails, requests are rerouted to the next closest one. Origin shielding and multiple origins add further resilience.**
- **Edge computing: functions such as AWS Lambda@Edge, CloudFront Functions and Cloudflare Workers run lightweight code at the edge to handle authentication, redirects, A/B tests, personalisation and request filtering without contacting the origin.**
- **Regional customisation: edge logic can localise language, currency and legal content, and can block or allow content by country.**
- **Lower data cost for users and operators: caching reduces the data transferred from the origin, saving bandwidth charges.**

3.5 Cloud Provider Support for Global Availability

A CDN handles content delivery, but a global app also needs its back-end logic and data close to users and resilient to failure. Cloud providers address this with a layered global infrastructure.

Building block	Description	Example services
Regions	Independent geographic areas, each with several data centres	AWS Mumbai, Azure Central India, Google Cloud asia-south1
Availability Zones	Isolated data centres within a region	Used for high availability inside a region
Edge locations / PoPs	Hundreds of small sites near end users	CloudFront, Azure Front Door, Cloud CDN
Global DNS and traffic routing	Latency-based, geolocation and failover routing	Amazon Route 53, Azure Traffic Manager
Global load balancing	Anycast IP that directs users to the nearest healthy region	Google Cloud Global Load Balancer, AWS Global Accelerator
Global databases	Multi-region replication of data	DynamoDB Global Tables, Cosmos DB, Cloud Spanner
Private global backbone	Provider-owned fibre network with fewer hops than the public internet	AWS, Azure and Google backbones

The app company can deploy identical stacks in several regions, for example Mumbai, Frankfurt and Virginia. DNS or a global load balancer routes each user to the closest region, and if a region fails traffic automatically moves to another. Data is replicated according to consistency needs, and regional data-residency rules can be respected by keeping certain users' data in specific regions.

3.6 Real-Time Streaming Application Example

Consider a video streaming app, StreamNow, launched in India, the Middle East, Europe and North America. The company stores master video files in cloud object storage in one primary region and encodes them into multiple bitrates. A CDN is placed in front of the storage.

- **On release day for a popular series, millions of viewers request the first episode. The first viewers in each region cause cache misses, and the edge fetches segments from the origin once.**
- **Within seconds the segments are cached and every later viewer in that region is served directly from the edge, so the origin sees only a tiny fraction of the traffic.**
- **Adaptive bitrate streaming adjusts quality for a user on a 4G connection in a small town versus a user on fibre broadband in a city.**
- **For live sports, the encoder pushes the stream to the CDN, which distributes short segments globally. Low-latency modes (LL-HLS or CMAF) reduce delay to a few seconds.**
- **Signed URLs and token authentication ensure that only subscribers can access the content, and geo-restriction applies licensing rules.**

3.7 Real-Time Gaming Application Example

In an online multiplayer game, latency (ping) directly affects fairness and enjoyment. Players expect round-trip times below roughly 50 to 80 milliseconds. Game companies use CDNs for large

and frequent downloads such as patches, textures and game assets, so that millions of players can update at the same time without delays. For the real-time match itself, game servers are deployed in many regions using services such as AWS GameLift, Azure PlayFab or Google Cloud Game Servers. A matchmaking service places players on the nearest game server, and a global accelerator or private backbone reduces jitter and packet loss between the player and the server. Edge compute can also handle leaderboards, authentication and anti-cheat checks near the player.

3.8 Challenges and Considerations

- **Cache invalidation: updated content must be refreshed quickly, using versioned file names or purge APIs.**
- **Dynamic and personalised content is harder to cache and may need edge compute or optimised routing.**
- **Cost: CDN data transfer and multi-region deployment add cost, which should be balanced with performance benefits.**
- **Data residency and privacy laws may limit where user data can be stored or replicated.**
- **Monitoring: real-user monitoring and synthetic tests per region verify that users are really getting the expected speed.**

3.9 Summary

CDNs and edge servers reduce latency by bringing content close to users, cache heavy assets to relieve the origin and give uniform performance across regions. Cloud providers extend this with global regions, traffic routing, replication and private backbones to deliver high availability. For streaming and gaming applications these capabilities are essential, as they turn a single-location service into a responsive worldwide platform.

Question 4: Compliance and Data Security for a Financial Institution

Question: ***A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.***

4.1 Introduction

Banks, insurance companies and payment firms handle some of the most sensitive data in the world: account numbers, card details, transaction histories and personal identity records. They are also among the most heavily regulated organisations. Moving to the cloud offers agility and cost advantages, but the institution must prove to regulators that its data remains confidential, accurate, available and located in permitted jurisdictions. Cloud providers support this through certified infrastructure, security tools and a clearly defined shared responsibility model.

4.2 Shared Responsibility Model

Security in the cloud is a partnership. The provider is responsible for the security of the cloud, which includes physical data centres, hardware, the hypervisor and core network. The customer is responsible for security in the cloud, which includes data classification, identity and access, encryption choices, application security, operating system patches on virtual machines and correct configuration. Many cloud breaches arise from customer misconfiguration, such as publicly exposed storage, rather than from provider failures.

Layer	IaaS	PaaS	SaaS
Physical and network	Provider	Provider	Provider
Operating system and runtime	Customer	Provider	Provider
Application	Customer	Customer	Provider
Identity and access	Customer	Customer	Customer
Data and its classification	Customer	Customer	Customer

4.3 How Cloud Providers Support Regulatory Compliance

Financial institutions must satisfy frameworks such as PCI DSS for card data, ISO/IEC 27001 for information security management, SOC 1, SOC 2 and SOC 3 for service controls, GDPR for personal data of EU residents, SOX for financial reporting, and national rules such as the RBI guidelines on outsourcing, cyber security and data localisation in India. Cloud providers help in the following ways.

- **Certified infrastructure: major providers are independently audited and hold certifications such as ISO 27001, PCI DSS Level 1, SOC reports and CSA STAR. The institution can inherit these controls for the provider-managed layers.**
- **Compliance portals: services like AWS Artifact, Azure Service Trust Portal and Google Compliance Reports Manager give on-demand access to audit reports and agreements.**
- **Compliance-focused services: tools such as AWS Config and Security Hub, Azure Policy and Defender for Cloud, and Google Security Command Center continuously compare resource settings with standards (CIS benchmarks, PCI DSS) and report drift.**
- **Policy as code: guardrails such as Service Control Policies or Azure Policy can prevent non-compliant actions, for example creating unencrypted storage or launching resources outside approved regions.**
- **Contractual commitments: data processing agreements, business associate or financial-services addenda, audit rights and breach-notification terms are available.**
- **Dedicated options: dedicated hosts, hardware security modules (HSM), private connectivity and sovereign or government cloud regions meet stricter needs.**
- **Third-party assurance: providers support regulator inspections and customer audits through documentation and evidence.**

4.4 Auditing

An audit asks three questions: what happened, who did it and was it allowed? Cloud platforms make this easier than traditional environments because every API call and configuration change can be recorded automatically.

- **Audit trails: AWS CloudTrail, Azure Activity Log and Google Cloud Audit Logs record who performed which action, when and from where.**
- **Configuration history: services keep a timeline of resource configuration so auditors can see what a system looked like on a past date.**
- **Evidence automation: compliance dashboards and reports reduce manual evidence collection for annual and ad-hoc audits.**
- **Tamper-resistant retention: logs are stored in immutable storage with long retention (often five to ten years for financial records) and restricted access.**

4.5 Role of Logging, Monitoring and Encryption in Security

Logging

Logs are the memory of the system. Application logs, operating system logs, network flow logs, DNS logs, database audit logs and identity logs together allow investigators to reconstruct events, detect misuse and prove compliance. Logs should be centralised in a separate, locked-down account so an intruder cannot erase them, time-synchronised and protected from tampering.

Monitoring and threat detection

Monitoring turns raw logs into alerts. Security Information and Event Management (SIEM) platforms such as Microsoft Sentinel, Splunk or Google Chronicle correlate events from many sources. Threat-detection services such as Amazon GuardDuty and Microsoft Defender use machine learning to spot unusual behaviour, for example a login from an unexpected country, a sudden large data download or an unusual pattern of payments. Real-time alerts and automated responses, such as disabling a compromised account or isolating a server, shorten the time between attack and containment.

Encryption

- **At rest: databases, disks and object storage use AES-256 encryption. Customer-managed keys in KMS or an HSM allow the institution to control rotation, revocation and access.**
- **In transit: TLS 1.2 or 1.3 protects all external and internal communication. Mutual TLS and VPN or private links protect service-to-service traffic.**
- **In use: confidential computing and secure enclaves protect data while it is being processed.**
- **Field-level encryption and tokenisation: card numbers and account identifiers can be replaced by tokens so most systems never see the real value, reducing PCI scope.**
- **Key separation: keys are kept apart from the data they protect, with split duties for key administrators.**

Identity and network controls

Strong identity management with MFA, least-privilege roles, just-in-time access and privileged-access monitoring supports the logging and encryption layers. Network segmentation, private subnets, firewalls, web application firewalls and DDoS protection reduce the attack surface. A zero-trust approach, which verifies every request regardless of its origin, is increasingly expected.

4.6 Ensuring Data Sovereignty

Data sovereignty means that data is subject to the laws of the country where it is stored or collected. Many jurisdictions restrict where financial data may be stored or processed. The Reserve Bank of India, for example, requires payment system data to be stored only in India. The institution can meet such requirements in several ways.

Measure	How it supports sovereignty
Select approved regions	Deploy and store data only in regions inside the permitted country, such as Mumbai or Hyderabad for India.
Region-lock policies	Use organisation-level policies to deny creation of resources in other regions.
Control replication and backup	Configure cross-region replication and disaster recovery only between approved regions.
Customer-managed keys in-country	Hold encryption keys in a local KMS or HSM so that data cannot be decrypted outside the jurisdiction.

Measure	How it supports sovereignty
Contractual guarantees	**Obtain written commitments on data location, sub-processors and government-access requests.**
Sovereign or government clouds	**Use dedicated regions operated under local legal control where required.**
Data classification and tagging	**Label data by sensitivity and residency so policy can be enforced automatically.**
Monitoring and audit	**Continuously check resource locations and keep evidence for regulators.**
Exit and portability plan	**Prepare for migration in case laws or provider terms change.**

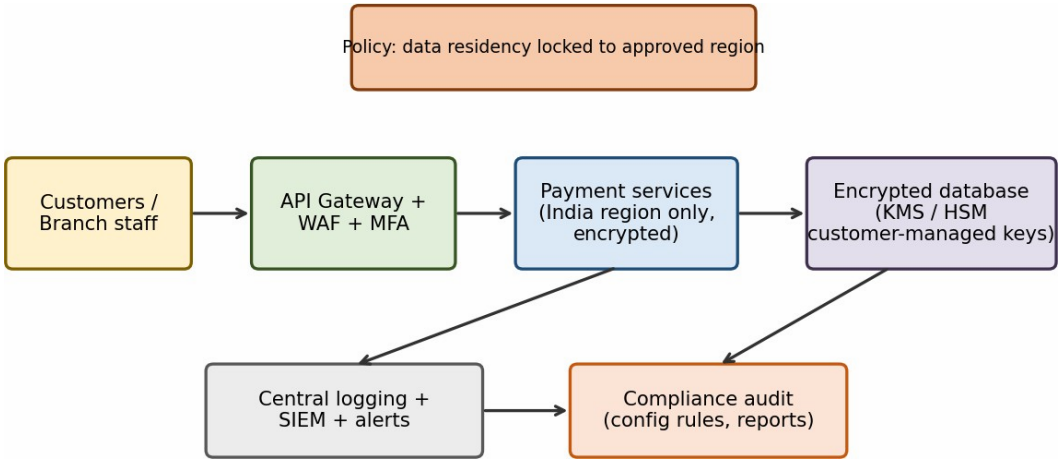


Figure 4: Compliant and auditable cloud architecture for financial transactions

4.7 Example: Sensitive Financial Transactions

Consider an Indian bank, SecureBank, that moves its mobile-payment platform to the cloud. A customer initiates a UPI or card transaction and the system must keep it confidential, correct and traceable.

Stage	Control applied
1. Request arrives	**The mobile app connects through an API gateway and WAF over TLS 1.3. The customer is authenticated using MFA and device binding.**
2. Processing	**Payment microservices run in private subnets in the Mumbai region only. Region-lock policy prevents any deployment elsewhere.**
3. Sensitive data handling	**The card number is tokenised, and the account data is encrypted using customer-managed keys held in an HSM in India.**
4. Storage	**The transaction is written to an encrypted database. Replication goes only to the Hyderabad region for resilience.**
5. Logging	**Every API call, database access and administrative action is logged to an immutable central store with retention for the period required by the regulator.**
6. Monitoring	**The SIEM correlates logs and the threat-detection service flags a suspicious burst of high-value transfers. An automated playbook blocks the account and alerts the fraud team.**

Stage	Control applied

7. Audit	**Compliance dashboards show PCI DSS and internal-policy checks passing. Auditors receive reports and evidence without any disruption to operations.**

In this design the bank keeps data in the required country, protects it with strong cryptography under its own control, detects fraud quickly and can demonstrate compliance at any moment.

4.8 Best Practices

- **Classify data first and apply the strongest controls to the most sensitive categories.**
- **Adopt a landing-zone design with separate accounts for production, logging and security.**
- **Automate compliance checks and remediation through policy as code.**
- **Run regular penetration tests, red-team exercises and incident-response drills.**
- **Perform due diligence on providers and manage concentration and exit risk.**

4.9 Summary

Cloud providers support financial compliance through certified infrastructure, audit tooling and contractual assurances. Logging, monitoring and encryption provide detection, accountability and confidentiality. Careful choice of regions, in-country keys and strong governance guarantee data sovereignty. Together they let a financial institution gain the benefits of the cloud without compromising on regulation or customer trust.

Question 5: Containerization, Orchestration and Microservices

Question: ***A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.***

5.1 Introduction

Traditionally, applications were installed directly on physical servers, and later inside virtual machines. Both methods suffer from the problem summarised as 'it works on my machine'. Differences in operating system versions, libraries and configuration between development, testing and production environments cause failures and delays. Containerization solves this problem by packaging an application together with its code, runtime, libraries and configuration in a standard unit called a container image. The same image runs identically on a laptop, a test server or any cloud. Docker made containers popular, and the Open Container Initiative (OCI) standardised their format.

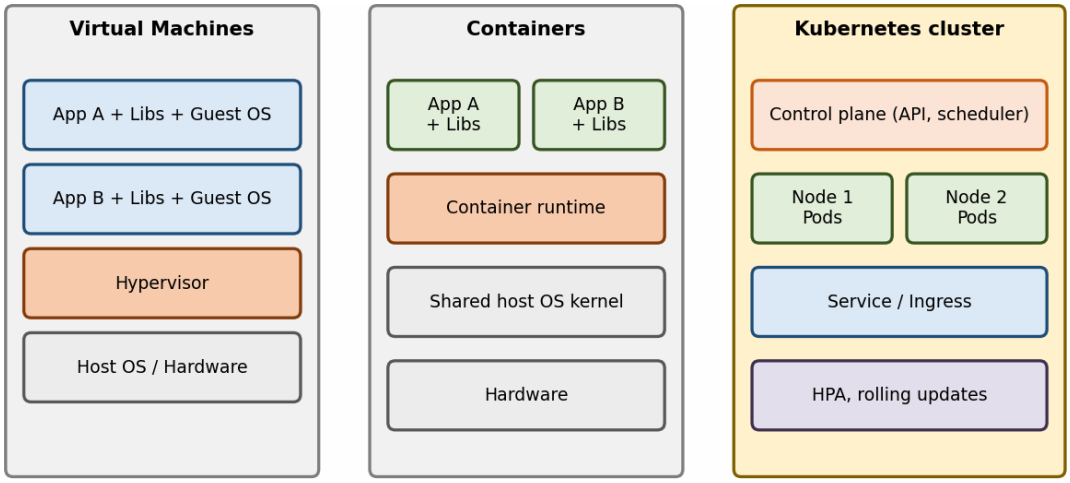


Figure 5: Virtual machines, containers and Kubernetes orchestration

5.2 Containers versus Virtual Machines

A virtual machine emulates a full computer. A hypervisor such as VMware ESXi, KVM or Hyper-V runs on the physical host and each VM contains its own guest operating system, binaries and application. A container virtualises at the operating-system level. All containers on a host share the host's kernel, and each container has only its application and dependencies, isolated by Linux namespaces and control groups (cgroups).

Aspect	Virtual machines	Containers
Isolation	Strong, hardware-level through hypervisor, separate kernel	Process-level, shared kernel; slightly weaker unless hardened
Size	Gigabytes, because each includes a full OS	Megabytes, because only app and libraries are included
Start-up time	Minutes	Seconds or milliseconds
Resource usage	High; each VM reserves CPU and memory for its guest OS	Low; resources are shared and allocated dynamically
Performance	Some overhead from virtualisation of hardware	Near-native performance
Density	Few VMs per host	Many containers per host, often several times more
Portability	Image tied to hypervisor format	Runs on any host with a container runtime
Operating system	Can run different OS types on the same host	Must match the host kernel type (Linux or Windows)
Typical use	Legacy applications, multiple OS needs, strong multi-tenant isolation	Microservices, cloud-native apps, CI/CD, scalable workloads
Scaling	Slow to clone and boot	Rapid scale out in seconds

In practice the two technologies are often combined. Containers frequently run inside VMs on public clouds, which provides strong tenant isolation from the provider and speed and efficiency from containers. Lightweight VM technologies such as Firecracker and Kata Containers aim to give VM-level isolation with container-like speed.

Performance and resource implications

- **Because no guest OS has to boot, a container starts in a fraction of a second, which enables fast auto-scaling and rapid deployment.**
- **Memory and CPU are not wasted on duplicate operating systems, so a server that runs ten VMs may run a hundred containers.**
- **Image layers are cached and shared, which saves storage and speeds up downloads.**
- **Resource limits (CPU shares and memory limits) can be applied per container, preventing noisy neighbours.**

5.3 Container Orchestration and Kubernetes

Running one container is simple. Running hundreds of containers across many servers, keeping them healthy, updating them without downtime and scaling them with demand is complex. Container orchestration automates this work. Kubernetes (K8s), originally developed by Google and now maintained by the Cloud Native Computing Foundation, is the leading orchestrator. Managed offerings include Amazon EKS, Azure AKS and Google GKE.

Key Kubernetes concepts

Concept	Purpose
Cluster, control plane and nodes	The control plane (API server, scheduler, controllers, etcd) manages worker nodes that run the workloads
Pod	The smallest deployable unit, containing one or more tightly coupled containers
Deployment and ReplicaSet	Declare the desired number of pod replicas and the image version; Kubernetes keeps reality equal to this declaration
Service	Gives a stable network address and load balancing to a changing set of pods
Ingress	Manages external HTTP and HTTPS access and routing
ConfigMap and Secret	Separate configuration and sensitive values from the image
Persistent Volume	Provides durable storage for stateful workloads
Namespace	Logical separation of teams, environments or applications

How orchestration simplifies deployment and scaling

- **Declarative deployment: the team describes the desired state in YAML files and Kubernetes works continuously to achieve it. This supports infrastructure as code and Git-based workflows (GitOps).**
- **Automated scheduling: the scheduler places containers on nodes with enough free resources, balancing the cluster.**
- **Self-healing: failed containers are restarted, unhealthy pods replaced and workloads rescheduled if a node dies.**
- **Horizontal Pod Autoscaler (HPA): increases or decreases pod replicas automatically based on CPU, memory or custom metrics, while the Cluster Autoscaler adds or removes nodes.**
- **Rolling updates and rollbacks: new versions replace old ones gradually with no downtime, and a faulty release can be rolled back with one command. Blue-green and canary strategies reduce risk further.**

- **Service discovery and load balancing: pods find each other by name, and traffic is distributed automatically.**
- **Resource management: requests, limits and quotas ensure fair use and protect critical services.**
- **Security and policy: role-based access control, network policies and secret management protect workloads.**
- **Ecosystem: tools like Helm for packaging, Prometheus for monitoring and service meshes like Istio add observability and traffic control.**

5.4 Portability Across Cloud Environments

Portability is one of the strongest reasons to adopt containers and Kubernetes.

- **Build once, run anywhere: a container image contains everything the app needs, so it behaves the same on a developer laptop, on-premises servers, AWS, Azure, Google Cloud or any other provider.**
- **Standard APIs: Kubernetes offers the same API and deployment files on every cloud, so skills, pipelines and manifests are reusable.**
- **Avoiding vendor lock-in: the organisation can move workloads or negotiate better pricing, since the application does not depend on a proprietary virtual-machine format.**
- **Hybrid and multi-cloud: workloads can run partly on-premises and partly in the cloud, or across several providers for resilience and regulatory reasons. Tools like Anthos, Azure Arc and Rancher help manage multiple clusters.**
- **Consistent environments: identical images across development, testing and production reduce environment-related bugs and make CI/CD pipelines reliable.**
- **Cloud bursting and disaster recovery: extra capacity or failover can be started in another cloud using the same images and manifests.**

Portability is not absolute. If the application uses provider-specific managed services such as proprietary databases or queues, those parts need adaptation. Using open standards, such as PostgreSQL, Kafka and S3-compatible APIs, and abstraction layers like Terraform, keeps the lock-in low.

5.5 Example of a Microservices-Based Application

In a microservices architecture, a large application is divided into small independent services, each responsible for one business function, owning its own data and communicating through APIs or messages. Containers are a natural fit because each service can be packaged, deployed and scaled independently.

Consider an online food-delivery application such as Swiggy or Zomato. It can be built from the following services, each running in its own container group on Kubernetes.

Microservice	Responsibility	Technology example	Scaling behaviour
API Gateway	Single entry, authentication, rate limiting	Kong or NGINX Ingress	Scales with total traffic
User Service	Registration, profile, addresses	Java Spring Boot, PostgreSQL	Moderate
Restaurant and Menu Service	Restaurant listings, menus, availability	Node.js, MongoDB	High read load, cached

Microservice	Responsibility	Technology example	Scaling behaviour
Search Service	Restaurant and dish search	Elasticsearch	Scales at lunch and dinner peaks
Order Service	Cart, order creation and status	Python, PostgreSQL	Scales strongly in the evening
Payment Service	Payment gateway integration	Go, encrypted store	Scales with checkout rate
Delivery Tracking Service	Live location of delivery partners	WebSockets, Redis	High real-time load
Notification Service	SMS, email and push messages	Event consumer on Kafka	Driven by queue length

When a customer places an order, the API gateway routes the request to the Order Service, which calls the Payment Service and publishes an 'order placed' event on a message queue. The Restaurant, Delivery and Notification services consume this event and perform their own tasks. If the Payment Service has a bug, only that service needs to be fixed and redeployed, and the rest of the application continues to run. During the dinner rush, the HPA scales the Order and Delivery Tracking services to many replicas, while the User Service stays small, which uses resources efficiently.

Benefits and challenges of microservices on containers

- **Benefits: independent deployments, fault isolation, technology freedom for each team, targeted scaling and faster release cycles through CI/CD.**
- **Challenges: more complex networking and debugging, need for distributed tracing and centralised logging, data consistency across services, and the operational skill required to run Kubernetes. Service meshes, observability tools and managed Kubernetes services help address these.**

5.6 Best Practices

- **Build small, minimal images and scan them for vulnerabilities, running containers as non-root users.**
- **Keep containers stateless and store state in managed databases or persistent volumes.**
- **Define resource requests and limits, liveness and readiness probes for every workload.**
- **Automate build, test and deployment with CI/CD and GitOps.**
- **Use secrets management, network policies and RBAC for security.**

5.7 Summary

Containers are lighter, faster and more resource-efficient than virtual machines, which makes them ideal for cloud-native software. Kubernetes automates deployment, scaling, healing and updates so teams can manage large fleets with little manual effort. Standard images and open APIs give portability across clouds, and microservices show how these tools combine to build resilient, independently scalable applications.

Overall Conclusion and References

Overall Conclusion

The five application scenarios show how core cloud capabilities solve real business problems. Elasticity through auto-scaling and load balancing keeps e-commerce platforms stable during

unpredictable demand. Durable, redundant and encrypted storage protects backup data at a lower cost than traditional systems. CDNs and global infrastructure deliver low-latency experiences to users everywhere. Compliance tooling, logging, encryption and region control let financial institutions meet strict regulation. Containers and Kubernetes provide efficient, portable and scalable deployment for modern microservices applications.

In each case, the benefits depend on correct design: stateless applications, least-privilege access, tested recovery plans, careful region selection and continuous monitoring. Cloud technology offers the tools, and good architecture and governance turn them into reliable, secure and cost-effective solutions.

Summary Table

Q	Scenario	Key cloud concepts	Example
1	Seasonal traffic spikes	Auto-scaling, load balancing, monitoring	Diwali sale on an e-commerce site
2	Secure backup storage	Durability, replication, encryption, IAM, lifecycle tiers	Ransomware recovery for a hospital group
3	Global app deployment	CDN, edge servers, regions, global routing	Video streaming and multiplayer gaming
4	Financial compliance	Shared responsibility, audit logs, SIEM, encryption, data residency	UPI and card payments for a bank
5	Containerization	Containers vs VMs, Kubernetes, portability, microservices	Food-delivery application

References

- **Amazon Web Services documentation: EC2 Auto Scaling, Elastic Load Balancing, Amazon S3, CloudFront, CloudTrail, AWS Artifact (docs.aws.amazon.com).**
- **Microsoft Azure documentation: Virtual Machine Scale Sets, Blob Storage redundancy, Azure Front Door, Azure Monitor, Azure Policy, AKS (learn.microsoft.com/azure).**
- **Google Cloud documentation: Managed Instance Groups, Cloud Storage, Cloud CDN, Cloud Audit Logs, GKE (cloud.google.com/docs).**
- **Kubernetes documentation: Concepts, Workloads and Horizontal Pod Autoscaling (kubernetes.io/docs).**
- **Docker documentation: What is a container? (docs.docker.com).**
- **Mell, P. and Grance, T. (2011). The NIST Definition of Cloud Computing. NIST Special Publication 800-145.**
- **Erl, T., Puttini, R. and Mahmood, Z. (2013). Cloud Computing: Concepts, Technology and Architecture. Prentice Hall.**
- **Newman, S. (2021). Building Microservices, 2nd Edition. O'Reilly Media.**
- **Reserve Bank of India. Storage of Payment System Data, circular on data localisation, 2018.**
- **PCI Security Standards Council. Payment Card Industry Data Security Standard (PCI DSS).**